AI Analysis of Draft Report

David Ewing (82171165)

1 June 2026

@ifundefined KOMAClassName @box @div @tempa @box+ @box @div @tempb @box @tempb @< @tempa @ @tempa @tempb @tempa @< @ @figure htbp , hidelinks, pdfcreator= LaTeX via pandoc *[formula]* Macro average F1 is used as the primary evaluation metric.
The validation split (2,000 tweets) serves as the test set, following the notebook template.
Because the training data is class-imbalanced — neutral tweets account for roughly 45 random undersampling is applied to reduce all classes to the size of the smallest (7,093 negative tweets), producing a balanced training set of 21,279 examples.
This approach was preferred over oversampling because it avoids synthesising or duplicating examples that may reinforce noisy labels.

Macro F1 therefore averages equally across all three classes, rather than being dominated by the majority class.
This is appropriate because the research question treats all three classes as equally important: a metric that concentrates weight on neutral tweets would obscure precisely the per-class performance differences this analysis is designed to detect.
Per-class precision and recall are also reported.

Accuracy is not used as the primary metric since it rewards over-predicting the majority class.
The dataset is the sentiment subset of the Cardiff NLP tweet _eval benchmark (Barbieri et al., 2020).
It was collected from Twitter and annotated through the SemEval 2017 Task 4A competition, where teams competed to classify tweet sentiment.

Labels — negative, neutral, and positive — were assigned by crowdsourced annotators, with final labels reflecting majority vote across multiple annotations.
Table 1.
Dataset split sizes and class distribution. *[formula]* & *[formula]* & *[formula]* & *[formula]* & *[formula]* Train (raw) & 45,615 & 7,093 & *[formula]*19,800 & *[formula]*18,700 Train (after undersampling) & 21,279 & 7,093 & 7,093 & 7,093 Validation / Test & 2,000 & 312 & 869 & 819 Test (held out) & 12,284 & — & — & — After applying random undersampling (random _state=82171165) to balance the training classes, the training set is reduced to 21,279 tweets — 7,093 per class.
The validation split (2,000 tweets) is used as the evaluation set.

The test split (12,284 tweets) is held out and not used in this report.

Exact per-class counts at each stage of processing are shown in notebook §2.5.

The class distribution in the validation set is unbalanced: 312 negative, 869 neutral, and 819 positive tweets.

This asymmetry — far fewer negative examples than neutral or positive — means per-class results need careful interpretation.
Poor performance on negative tweets may partly reflect fewer test examples as well as weaker training signal.
Tweets are short, informal, and noisy: hashtags, @-mentions, URLs, slang, emoji, and abbreviations are common.
They are also highly context-dependent — the same words can carry different sentiment depending on the event being discussed or who is being addressed.

These properties make sentiment classification in Twitter data intrinsically harder than in product reviews or news text.

Two classifiers are evaluated: logistic regression and a shallow decision tree.

Both use the same scikit-learn pipeline (notebook section 3), fitted to the balanced training set and evaluated on the validation split.

The pipeline has four steps.
First, a TextCleaner component strips whitespace.
Second, a SpacyPreprocessor tokenises each tweet using the en _core _web _sm spaCy model and caches the results in an SQLite feature store (assignment-sentiment.sqlite).
Third, a FeatureUnion assembles the feature matrix.

In this initial run the only active feature block is a TokensVectorizer: unigram token counts for the top 100 most frequent tokens by document frequency, after removing stop words and punctuation, with lowercasing applied.
Fourth, the feature matrix is passed to the classifier.
Logistic regression is configured with max _iter=5000, random _state=42.

The decision tree uses max _depth=3, random _state=42.
The shallow depth restricts the tree to at most seven leaf nodes, forcing selection of only the most discriminating tokens and producing a human-interpretable model.
Both classifiers share identical preprocessing, achieved by calling set _params(classifier=clf) inside a loop over both configurations.

Dummy-image Figure 1.
Scikit-learn pipeline.
This run uses a minimal, interpretable baseline feature set.

The model plan lists additional feature types to explore in subsequent runs: TF-IDF weighted unigrams with a larger vocabulary, bigrams, VADER sentiment lexicon counts, document-level text statistics (tweet length, punctuation counts), and POS tag sequences.
A binary task (negative vs positive only) is also planned.
Tables 2 and 3 show the classification report for logistic regression and decision tree respectively, on the 2,000-tweet validation split.
Figures 2 and 3 show the corresponding confusion matrices.

Figure 4 shows the top LR feature coefficients per class.
Figure 5 shows the decision tree structure.
Table 2.

Logistic Regression — classification report (unigram counts, top 100 features). *[formula]* & *[formula]* & *[formula]* & *[formula]* & *[formula]* negative & 0.245 & 0.446 & 0.316 & 312 neutral & 0.517 & 0.534 & 0.525 & 869 positive & 0.631 & 0.413 & 0.499 & 819 macro avg & 0.464 & 0.464 & 0.447 & 2000 accuracy & — & — & 0.470 & 2000 Table 3.
Decision Tree (max _depth=3) — classification report (unigram counts, top 100 features). *[formula]* & *[formula]* & *[formula]* & *[formula]* & *[formula]* negative & 0.000 & 0.000 & 0.000 & 312 neutral & 0.454 & 0.978 & 0.620 & 869 positive & 0.798 & 0.121 & 0.210 & 819 macro avg & 0.417 & 0.366 & 0.277 & 2000 accuracy & — & — & 0.474 & 2000 Table 4.
Macro F1 comparison: Logistic Regression vs Decision Tree. *[formula]* & *[formula]* & *[formula]* & *[formula]* negative & 0.316 & 0.000 & +0.316 neutral & 0.525 & 0.620 & −0.095 positive & 0.499 & 0.210 & +0.289 macro & 0.447 & 0.277 & +0.170 Figure 2.
Logistic Regression — confusion matrix.

Figure 3.
Decision Tree — confusion matrix.
Figure 4.

Logistic Regression — top 20 feature coefficients per class.
Figure 5.
Decision Tree visualisation (max _depth=3).
The logistic regression baseline achieved a macro F1 of 0.447.

This is below 0.5, a rough reference point for a classifier that handles all three classes with at least some reliability.
The result is consistent with the known difficulty of three-class sentiment classification in tweets: the task is harder than binary sentiment in longer, more formal text, and the class boundaries in Twitter data are inherently fuzzy.
The decision tree achieved a lower macro F1 of 0.277, despite marginally higher accuracy (0.474 vs 0.470).

The accuracy paradox is instructive: the test set contains 869 neutral tweets (43.5 predicts neutral for almost all inputs achieves near-50 producing a very low macro F1.
This confirms that macro F1 is the correct metric for this dataset.
The most striking pattern in the LR results is the asymmetry across classes.

Positive tweets achieved the highest F1 (0.499), driven by high precision (0.631) but only moderate recall (0.413): the model is conservative about predicting positive, but when it does it is often right.
Negative tweets achieved the lowest F1 (0.316), with very low precision (0.245) but moderate recall (0.446): the model over-predicts negative, and most tweets predicted as negative were actually neutral or positive.
Neutral sat in the middle (F1=0.525).

The fact that neutral is not the worst-performing class is somewhat surprising.
The conventional view — supported by the assignment notes and by Kenyon-Dean et al. (2018) — is that neutral is the hardest class because it lacks distinctive lexical markers.
The LR results do not clearly support this: neutral F1 (0.525) exceeds negative F1 (0.316).

This may be because the neutral class is the largest in the test set (869 examples), giving the model more evaluation signal, or because neutral tweets tend to contain common, non-sentiment-loaded words that are reliably represented in the top-100 unigram vocabulary.
Negative tweets may use more varied and context-dependent language, and the test set has only 312 negative examples — the fewest of the three classes.
The decision tree result tells a clearer story.

With max _depth=3 and 100 unigram count features, the tree predicted neutral for almost all inputs (recall=0.978 for neutral, near-zero for both other classes).
This is consistent with a shallow tree learning a dominant majority-class heuristic — even though training data was balanced, the learned split rules apparently generalise better for neutral than for the other classes.
The central question of this project is not simply ”how good is the classifier?” but ”what does classifier performance tell us about the labels?” The low macro F1 is partly attributable to model limitations — the feature set is minimal — but it is also consistent with genuine label noise in the dataset.

The most suggestive evidence comes from the negative class.
Low precision (0.245) means the model is making many false positive predictions for negative sentiment.
From a labelling perspective: if the model predicts negative based on reasonable lexical cues but the gold label is neutral, this may indicate cases where annotators labelled a borderline tweet as neutral rather than negative.

The SemEval annotation guidelines used neutral as a default for tweets that did not clearly express positive or negative sentiment — which may have produced a diffuse, underspecified neutral category that overlaps substantially with mild negative expression.
Kenyon-Dean et al. (2018) argue that sentiment classification in tweets is ”complicated” precisely because sentiment is a property of the author s relationship to the content, not just of the words used.
The same phrase (”not bad at all”) may be positive or ironic depending on context.

Unigram count features, which discard word order, negation scope, and context, are poorly equipped to capture these distinctions.
The weak results are therefore expected — but they also suggest that the hard cases may involve genuine ambiguity that even human annotators would disagree on.
This analysis is preliminary.

Only one feature configuration has been evaluated — 100 unigram count features — and no comparison across feature types has been made.
The planned additional experiments (VADER features, binary task, bigrams, text statistics) are not yet complete.
The decision tree result cannot be fully interpreted without examining the tree visualisation (Figure 5, currently a placeholder).

The misclassification analysis in notebook section 4.2 was run but individual tweet examples have not yet been examined in detail.
The binary task experiment (negative vs positive only) is needed to determine whether neutral is primarily responsible for the overall weak performance.
Until this is run, the relative contribution of the neutral class cannot be quantified.
The analysis is also constrained to the validation split.

The held-out test split (12,284 tweets) has not been used and would provide a more reliable estimate of generalisation performance.
Barbieri, F., Camacho-Collados, J., Espinosa-Anke, L., & Neves, L. (2020).
TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification.

In Findings of the Association for Computational Linguistics: EMNLP 2020 (pp. 1644–1650). https://doi.org/10.18653/v1/2020.findings-emnlp.148
Hutto, C.

J., & Gilbert, E. (2014).

VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text.

In Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media (pp. 216–225).

Kenyon-Dean, K., Ahmed, E., Bhosale, S., Brooks, B., Laframboise, C., Roper, F., … & Singh, S. (2018).
Sentiment Analysis: It s Complicated!
In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (pp. 1886–1895). https://doi.org/10.18653/v1/N18-1171 Rosenthal, S., Farra, N., & Nakov, P. (2017).

SemEval-2017 Task 4: Sentiment Analysis in Twitter.

In Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017) (pp. 502–518).

Appendix